# Assignment 2: Full Auth.js Implementation + AI Chat Interface

**Due Date:** Sunday 17th August (group task)
**Presentation:** Required at completion

## 1 Project Overview

This assignment is a **full redo** of Assignment 1's authentication application. You must implement **all features** end-to-end (none are optional) and integrate an **AI-powered chat interface** using the `Vercel AI SDK` with a `Gemini API Key`. If you get stuck, **reach out on Slack** promptly.

## 2 Team Groupings

- Sheraz Ahmed & Muhammad Hanzla
- Hammadullah Abid & Mahad Khalid Ghafoor
- Muhammad Faseeh & Abdul Aziz
- Choudhary Muhammad Junaid & Muhammad Muzamil
- Muhammad Ahmad & Behram Khan
- Shirmeen Aamir & Haris Ahmed & Abdullah Tahir
- Inzish Khan & Hammad Ali

## 3 Required Features

### 3.1 Authentication & Authorization

- **Auth.js with email + password**
- **Database sessions (PostgreSQL)** — no JWT
- **Protected routes** (dashboard and any user-only pages)
- **Profile management** (view & update)
- **Admin role & dashboard**:
  - View all registered users
  - Basic user analytics/statistics
  - Admin controls (e.g., role change/disable)

### 3.2 Database & ORM

- **PostgreSQL** with `Drizzle ORM`
- Proper use of **drizzle.config.ts**
- Use **Drizzle Studio** (no PGAdmin)

### 3.3 OAuth & Email Flows

- **OAuth:** Google & GitHub sign-in
- **Email verification** during signup
- **Password reset** via secure email link

### 3.4 AI Chat Interface

- **Vercel AI SDK** with **Gemini API Key**
- SvelteKit + TailwindCSS **chat UI**
- Must handle: user input, response display, loading states, errors
- Encapsulate logic into reusable modules/components

## 4 Source Control & Workflow (Required)

- **Branching:** Work on `feature/{short-name}` branches. No direct commits to `main`.
- **Pull Requests:** Open PRs to `main`. At least one peer review within the group.
- **Commit Quality:** **Working commits only**. No broken builds. Use **Conventional Commits** (`feat:`, `fix:`, `chore:`, `refactor:`).
- **Commit Frequency:** Small, incremental commits (~ 5–15 changes per commit) with clear messages.
- **README & .env.example:** Keep up-to-date with every PR if setup changes.

## 5 Fresh-Clone Validation (Required)

Your repo must pass a clean setup by a new developer **without manual fixes**. Reviewers will:

1. **Clone fresh** on a new machine/environment.
2. Run `cp .env.example .env` and fill only required secrets.
3. Run `pnpm db:start` (Postgres).
4. Run `pnpm install`, `pnpm db:push`, then `pnpm dev`.

## 6 Extensions

If you finish early, pick **one** extension to implement:

- **Streaming:** Stream AI responses instead of returning them all at once.
- **Database Chat Storage:** Save chat history per user in PostgreSQL.

### Super Extension

- **Tree-Structured Chat History:** Store chat messages in a tree format so users can fork a conversation from any message.

## 7 Deliverables

### 7.1 Working Application

- Auth.js sessions, protected routes, profile
- Admin dashboard with role management & analytics
- OAuth (Google & GitHub), email verification, password reset
- AI chat interface (Vercel AI SDK + Gemini)
- Responsive UI (TailwindCSS)

### 7.2 Repository

- Public GitHub repo with CI passing badge
- Clean commit history (no broken commits on main)
- `README.md` with full setup & troubleshooting
- `.env.example` with all required variables

### 7.3 Presentation & Demo

- 10 minutes total
- Cover: Auth & RBAC (Role Based Access Control), Drizzle, OAuth/email flows, AI chat, and key challenges

## 8 Success Criteria

✓ All required features implemented (none omitted)

✓ Passing CI on every PR and on `main`

✓ Fresh-clone validation succeeds without manual intervention

✓ Extensions implemented if possible

✓ Clear source control hygiene and working commits

✓ README enables any dev to run the project quickly

---

*Aim for production-readiness: correctness, reliability, and ease of setup. Ask questions on Slack early.*